

Actividad N°2: Tópicos Avanzados III

Optimización paralela del problema de la mochila

Benjamín Borgeaud

bborgeaud2021@alu.uct.cl

Sebastián Cisternas

scisternas2020@alu.uct.cl

Eliaser Concha

econcha2019@alu.uct.cl

Christián Ferrer

cferrer2021@alu.uct.cl

Patricio Valdes

pvaldes2020@alu.uct.cl

10 de mayo de 2026

1. Introducción y contexto del problema

El problema de la mochila se define como un modelo matemático de optimización, dónde se recibe un conjunto de elementos con pesos y valores específicos, y se debe determinar qué cantidad de cada elemento se debe incluir en una colección. El objetivo de este tipo de modelos busca maximizar el valor total acumulado sin exceder un límite de peso predefinido conocido como la capacidad de la mochila.

Para el desarrollo de este proyecto, se trabaja con una versión extendida del problema de la mochila cuyo modelo es la base para la resolución de problemas críticos en diversas industrias:

- Finanzas: este tipo de modelos permite seleccionar combinaciones de inversiones o activos para maximizar ganancias bajo restricciones en presupuesto y riesgos.
- Logística: permite determinar qué tipo de productos se deben cargar en camiones, barcos o aviones considerando sus respectivos límites de peso y volumen.
- Optimización de recursos: permite decidir sobre la distribución de recursos limitados como dinero, tiempo o espacio con la finalidad de maximizar beneficios o rendimiento.

El problema de la mochila analizado desde el punto de vista temporal presenta una complejidad de tipo exponencial en situaciones dónde se resuelve mediante fuerza bruta, ya que al existir 2 opciones para cada objeto da origen a que el número total de combinaciones tiende a crecer como 2^n resultando con una complejidad de $O(2^n)$. Sin embargo, al aumentar la cantidad de objetos, su tiempo de ejecución crece de forma rápida dificultando su resolución para entradas grandes. Es por este comportamiento que el problema de la mochila en sí, es clasificado como NP-hard gracias a la ausencia de un algoritmo polinomial que sea capaz de resolver de manera óptima para determinados casos dónde se requiere una cantidad de tiempo extremadamente alta cuando el número de entradas aumenta considerablemente.

Debido a la complejidad exponencial del problema de la mochila, el trabajo en específico se relaciona con el paralelismo ya que requiere dividirse entre múltiples núcleos del procesador para permitir que distintas combinaciones sean evaluados de manera simultánea, dónde la función del paralelismo es reducir el tiempo total de ejecución distribuyendo carga entre varios procesos concurrentes.

2. Formulación matemática extendida

2.1. Variables

Para el modelo extendido, se precisa formular un vector binario $X = \{x_1, x_2, \dots, x_n\}$ de capacidad n , donde n corresponde al número total de items disponibles en la instancia. Para cada item, $i \in [1, n]$, dada la siguiente función ramificada:

$$x_i = \begin{cases} 1, & \text{si el item } i \text{ es seleccionado para la mochila,} \\ 0, & \text{en caso contrario.} \end{cases}$$

Dentro de cada item, se consideran los llamados parámetros de entrada considerados como los atributos extraídos de la instancia. Dónde:

- v_i corresponde al valor o beneficio asociado
- w_i corresponde al peso del item
- vol_i corresponde al volumen del item
- cat_i corresponde a la categoría de pertenencia

2.2. Función objetivo

El objetivo es maximizar la aptitud o fitness total de la solución. Definiéndose como el valor total de los ítems seleccionados, penalizado por el incumplimiento de las restricciones dadas. La función objetivo viene dada por:

$$\text{Maximizar } f(x) = \sum_{i=1}^n v_i x_i - \text{PenalizacionTotal}(X)$$

Dónde, $\text{PenalizacionTotal}(X)$ corresponde a la suma ponderada de las violaciones a las restricciones duras y blandas, con la finalidad de permitir que el espacio de búsqueda sea guiado hacia la factibilidad.

2.3. Restricciones extendidas

2.3.1. Restricciones duras

Las restricciones duras son aquellas que la solución final reportada debe cumplir obligatoriamente. El incumplimiento de cualquiera de ellas invalida la solución, por lo que se penalizan con un valor suficientemente alto para que ningún individuo inválido compita en igualdad de condiciones con soluciones factibles.

1. **Restricción de peso:** la suma de los pesos de los ítems seleccionados no debe superar la capacidad máxima W :

$$\sum_{i=1}^n w_i x_i \leq W$$

2. **Restricción de volumen:** la suma de los volúmenes no debe superar la capacidad máxima V :

$$\sum_{i=1}^n vol_i x_i \leq V$$

3. **Incompatibilidades:** si el par (a, b) es incompatible, no pueden seleccionarse simultáneamente:

$$x_a + x_b \leq 1 \quad \forall (a, b) \in \mathcal{I}$$

4. **Dependencias:** si se selecciona el ítem a y este requiere el ítem b , entonces b también debe estar seleccionado:

$$x_a \leq x_b \quad \forall (a, b) \in \mathcal{D}$$

2.3.2. Restricciones blandas

Las restricciones de categoría son tratadas como blandas: el algoritmo genético puede explorar soluciones que las violen temporalmente, pero reciben una penalización proporcional al grado de violación. Esto permite que el algoritmo explore regiones cercanas a la factibilidad sin descartar inmediatamente individuos que pueden mejorarse mediante cruzamiento o mutación.

1. **Restricciones por categoría:** para cada categoría $c \in \mathcal{C}$ con mínimo $\min_c$ y máximo $\max_c$:

$$\min_c \leq \sum_{i: cat_i=c} x_i \leq \max_c$$

La violación se mide como el exceso sobre el máximo más el déficit bajo el mínimo.

2.4. Función de aptitud y penalizaciones

La función de aptitud integra el valor total de la solución menos las penalizaciones ponderadas por tipo de restricción:

$$\begin{aligned} f(X) = & \sum_{i=1}^n v_i x_i - \alpha \cdot \text{exceso_peso}(X) - \beta \cdot \text{exceso_vol}(X) \\ & - \delta \cdot \text{incompatibilidades}(X) - \varepsilon \cdot \text{dependencias}(X) - \gamma \cdot \text{viol_categoría}(X) \end{aligned} \quad (1)$$

Los parámetros se calibran automáticamente a partir de la instancia:

$$\alpha = \beta = \delta = \varepsilon = 10 \times v_{\text{máx}} \times n \quad \gamma = \bar{v}/2$$

donde $v_{\text{máx}}$ es el valor máximo de un ítem y $\bar{v}$ es el valor promedio. Esta calibración garantiza que cualquier violación de restricción dura supera el máximo beneficio teórico posible, haciendo imposible que un individuo inválido presente un fitness positivo competitivo.

2.5. Criterio de factibilidad

Una solución X es factible si y solo si:

$$\sum_{i=1}^n w_i x_i \leq W \wedge \sum_{i=1}^n vol_i x_i \leq V \wedge (a, b) \in \mathcal{I} : x_a = x_b = 1 \wedge (a, b) \in \mathcal{D} : x_a = 1 \wedge x_b = 0$$

Las violaciones de categoría no invalidan la factibilidad. La solución final reportada siempre corresponde al mejor individuo factible encontrado en toda la historia de la ejecución. Si el individuo de mayor fitness global viola alguna restricción dura, se reporta en su lugar la mejor solución factible acumulada.

3. Descripción del algoritmo genético

3.1. Variante 1: Algoritmo genético secuencial (línea base)

La variante secuencial corresponde a la implementación base del algoritmo genético: ejecución completamente en un único hilo, sin ningún mecanismo de paralelismo ni subpoblaciones. Sirve como referencia para calcular speedup y eficiencia de las variantes paralelas. Todos los parámetros (población, generaciones, semilla, operadores) son idénticos a las variantes paralelas para que la comparación sea justa.

3.2. Descripción general del algoritmo genético

El algoritmo genético es una técnica de búsqueda y optimización basado en el proceso de selección natural y la genética biológica. Es utilizado para resolver determinados problemas dónde el espacio de búsqueda es tan grande que no se puede explorar de manera exhaustiva.

El algoritmo trata a las posibles soluciones como individuos de una población cuyo objetivo es competir por sobrevivir. La idea es que los individuos más aptos tienen una mayor probabilidad de reproducirse y transmitir sus características a la siguiente generación.

Para el contexto del problema de la mochila, el algoritmo utiliza una representación binaria comparado como un cromosoma de bits. Dónde cada individuo es representado como un vector cuya posición denominado gen representa un ítem de la instancia.

Un concepto relevante de este algoritmo es la población, la cual se define como un conjunto de soluciones candidatas que evolucionan simultáneamente. Una población posee un tamaño, una inicialización y una evolución. En la inicialización, se crean los individuos al azar que permite asegurar que el algoritmo explore diferentes regiones del espacio de búsqueda desde el inicio. La evolución es una característica no estática que se desarrolla a través de las generaciones y determina que los individuos de baja calidad sean reemplazados por descendientes de individuos más aptos.

En cuanto a la selección, es un mecanismo que decide qué individuos son aptos para reproducirse, para ello se utiliza la estrategia de selección por torneo. El proceso consiste en seleccionar de manera aleatoria un grupo de individuos de la población. Dónde, el individuo que tiene un mejor fitness es elegido para formar parte de los progenitores. El método se debe ejecutar dos veces para obtener la pareja que realizará el cruzamiento.

La función de aptitud o fitness es una métrica que define qué tan buena es una solución. Dentro de sus componentes, se calcula como el valor total de los ítems menos una penalización vinculada a las violaciones detectadas. Su importancia radica en determinar los individuos con mayor probabilidad de ser progenitores o padres.

El cruzamiento es el mecanismo principal de búsqueda que permite combinar información de dos genes padres para crear un individuo cuyo potencial pueda superar a la de sus progenitores. Para ello, se implementa el cruce de punto o crossover el cual corta el ADN de los padres en un punto aleatorio y se intercambian sus mitades.

La mutación es un mecanismo de exploración que introduce cambios de manera aleatoria y de forma pequeña en ciertos individuos, y corresponde a una mutación del tipo de inversión de bit cuyo fin es asignar una probabilidad para que cada gen del cromosoma cambie de estado. Esa probabilidad se determina por un ratio bajo como por ejemplo 2 % con la finalidad de mantener la diversidad genética y evitar convergencia prematura.

El elitismo se define como una política de supervivencia del individuo más apto para garantizar que la calidad del individuo no disminuya con el paso de las generaciones. Bajo esta premisa, el algoritmo permite identificar los mejores individuos de la población actual. Al identificarlos se copian de forma directa a la nueva generación antes de aplicar cruzamiento o mutación sobre el resto de la población.

Con respecto al criterio de término, el algoritmo finaliza dado un número fijo de iteraciones o generaciones. Se optó deliberadamente por el criterio de **generaciones máximas únicamente**

(`max_gen = 500`), descartando un criterio de convergencia como parada anticipada. Esta decisión garantiza que todas las variantes (secuencial, paralela e islas) ejecuten exactamente el mismo número de generaciones, haciendo que las comparaciones de tiempo y calidad sean directamente comparables. Finalmente, el algoritmo entrega la mejor solución factible (cumpliendo todas las restricciones duras) encontrada durante toda la historia de la evolución.

Parámetros utilizados:

- `pop_size`: cantidad de individuos en la población
- `max_gen`: límite de iteraciones del algoritmo
- `mutation_rate`: probabilidad de mutar cada gen
- `crossover_rate`: probabilidad de que dos progenitores intercambien genes
- `elite_size`: número de mejores individuos que sobreviven de forma automática
- `tournament_size`: cantidad de competidores en cada torneo de selección
- `num_threads`: número de hilos para ejecución paralela con OpenMP

4. Estrategias de paralelización

4.1. Variante 2: Evaluación paralela de la población (OpenMP)

La paralelización de la variante paralela se concentra en dos zonas del bucle evolutivo: la **evaluación de aptitud** y la **generación de descendencia**.

4.1.1. Evaluación de aptitud (`evaluatePopulation`)

La evaluación es la operación más costosa del algoritmo ya que requiere recorrer el cromosoma completo para calcular peso, volumen, incompatibilidades, dependencias y valor. Dado que cada individuo i de la población se evalúa de forma completamente independiente (lee `population_[i]`, escribe `fitness_values_[i]` sin compartir índices), el bucle es paralizabile sin condiciones de carrera:

```
#pragma omp parallel for schedule(dynamic) \
    shared(population_, fitness_values_, inst_, penalty_) \
    firstprivate(pop) default(none)
for (int i = 0; i < pop; ++i)
    fitness_values_[i] = computeFitness(population_[i], inst_, penalty_);
```

- **Datos compartidos (solo lectura):** `population_`, `inst_`, `penalty_`. Al no escribirse durante el loop, no hay condición de carrera.
- **Datos privados por iteración:** `fitness_values_[i]` — cada hilo escribe en un índice exclusivo.
- **Schedule dynamic:** se usa `schedule(dynamic)` para balancear la carga cuando distintos cromosomas tienen longitudes de evaluación variables (p.ej., instancias con muchas incompatibilidades activan más ramas del cálculo).

4.1.2. Generación de descendencia (runOneGeneration)

Los operadores de cruzamiento, mutación y selección por torneo se ejecutan en paralelo para generar la mitad de la nueva población. El principal riesgo es el uso compartido del generador de números aleatorios (RNG), que es un estado mutable. La solución adoptada es asignar un RNG `thread_local` a cada hilo, sembrado con la semilla base más el identificador de hilo y el número de generación:

```
std::mt19937 local_rng(params_.seed + omp_get_thread_num()
                        + generation_ * 1000);
#pragma omp for schedule(static)
for (int pair = 0; pair < pairs_needed; ++pair) {
    int p1 = tournamentSelect(local_rng);
    int p2 = tournamentSelect(local_rng);
    auto [c1, c2] = crossover(population_[p1], population_[p2], local_rng)
    ;
    mutate(c1, local_rng);    mutate(c2, local_rng);
    newPop[elite + pair*2]    = c1;
    newPop[elite + pair*2+1] = c2;
}
```

- **Datos compartidos (solo lectura):** `population_` y `fitness_values_` son leídos durante la selección, nunca escritos.
- **Datos privados:** `local_rng`, `c1`, `c2`.
- **Escritura en newPop:** cada hilo escribe en posiciones disjuntas (`elite + pair*2` y `elite + pair*2 + 1`), por lo que no existe condición de carrera.
- **Elitismo secuencial:** los mejores `elite_size` individuos se copian antes del bucle paralelo para garantizar que no sean sobreescritos.
- **Schedule static:** se usa `schedule(static)` ya que cada par de hijos tiene costo uniforme, minimizando overhead de sincronización.

4.2. Variante 3: Modelo de islas

El modelo de islas divide la población en $N = 4$ subpoblaciones (islas) que evolucionan de forma independiente y realizan migraciones periódicas en topología de anillo.

4.2.1. Configuración

Parámetro	Valor
Número de islas (N)	4
Tamaño de población por isla	200 individuos
Total de generaciones	500
Intervalo de migración	cada 25 generaciones
Individuos migrantes por isla (k)	2 (los mejores por fitness)
Topología	Anillo ($i \rightarrow (i + 1) \bmod N$)
Semilla por isla	<code>seed_base + i × 997</code>

Cuadro 1: Parámetros del modelo de islas.

*En el diseño experimental se varió N igual al número de hilos (1, 2, 4, 8) para evaluar el impacto del número de subpoblaciones. La tabla muestra la configuración canónica de 4 islas.

4.2.2. Paralelismo y sincronización

Cada isla evoluciona en su propio hilo mediante `#pragma omp parallel for num_threads(N) schedule(static)`. Al ser poblaciones completamente independientes (no comparten memoria de individuos), no existe condición de carrera durante la evolución:

```
#pragma omp parallel for num_threads(n) schedule(static)
for (int i = 0; i < n; ++i)
    for (int s = 0; s < steps; ++s)
        islands[i].runOneGeneration();
// Barrier implícito al salir del parallel for
migrate(islands);
```

La migración se ejecuta de forma secuencial tras el *barrier* implícito al finalizar el bloque paralelo. Para evitar que una isla reciba migrantes de su vecina antes de que esta haya exportado los propios, la migración utiliza un buffer de dos fases:

1. **Fase 1:** cada isla i exporta sus k mejores individuos al buffer `buffers` (solo lectura de las islas).
2. **Fase 2:** se inyectan los individuos: la isla $(i + 1) \bmod N$ recibe `buffers`.

4.2.3. Justificación de parámetros

El intervalo de 25 generaciones permite que cada isla converja suficientemente antes de recibir material genético externo, evitando homogeneización prematura. El criterio de $k = 2$ migrantes (los mejores) equilibra presión selectiva y diversidad. La topología de anillo garantiza que la información de buena calidad se propague gradualmente sin concentrarse en una sola isla.

4.2.4. Overhead y comportamiento del speedup

El modelo de islas no está diseñado para reducir el tiempo de cómputo sino para mejorar la **calidad de la solución**. Dado que cada isla ejecuta 500 generaciones completas con una población de 200 individuos, el cómputo total es mayor que en la variante secuencial con la misma configuración. El overhead de migración y la sincronización entre islas explican por qué el speedup del modelo de islas es inferior a 1 (mayor tiempo que el secuencial), mientras que la calidad de solución es consistentemente superior.

5. Diseño experimental

5.1. Configuración del entorno

Los experimentos se ejecutaron en un equipo con las siguientes características de hardware:

Característica	Valor
CPU	AMD Ryzen 5 5600X
Núcleos físicos / hilos	6 / 12
RAM	32 GB DDR4 3600 MHz
Almacenamiento	SSD NVMe
Sistema operativo	Windows 11
Compilador	MinGW-w64 g++ (OpenMP 4.5)

Cuadro 2: Configuración del equipo de prueba (Christian Ferrer).

5.2. Instancias utilizadas

Se generaron tres instancias del problema mediante el generador incluido en el proyecto:

Instancia	Ítems	W	V
Pequeña (small)	100	$0,40 \times \sum w_i$	$0,40 \times \sum vol_i$
Mediana (medium)	1.000	$0,40 \times \sum w_i$	$0,40 \times \sum vol_i$
Grande (large)	10.000	$0,40 \times \sum w_i$	$0,40 \times \sum vol_i$

Cuadro 3: Tamaños de instancia. Las capacidades se fijan al 40 % del total disponible.

Cada instancia contiene ítems con valor, peso, volumen y categoría generados aleatoriamente. Se incluyen reglas de categoría, incompatibilidades entre pares de ítems y dependencias entre ítems para activar todas las restricciones del modelo extendido.

5.3. Variantes y configuraciones

Se ejecutaron tres variantes para cada combinación de instancia e hilos:

Variante	Hilos	Descripción
<code>sequential</code>	1	Línea base (Variante 1), completamente secuencial
<code>parallel</code>	1, 2, 4, 8	OpenMP, fitness + operadores paralelos (Variante 2)
<code>islands</code>	1, 2, 4, 8	Modelo de islas con N islas = hilos (Variante 3)

Cuadro 4: Variantes experimentales. Para `islands`, el número de islas se fija igual al número de hilos (1, 2, 4 u 8) para evaluar el impacto de N . En total: 3 instancias $\times$ 9 configuraciones = 27 grupos.

5.4. Protocolo de repetición y semillas

Cada configuración se ejecutó con **15 repeticiones** usando semillas fijas $\{1, 2, \dots, 15\}$, registradas en `results/resultados.c` para garantizar reproducibilidad completa. El total de ejecuciones fue $27 \times 15 = 405$.

5.5. Parámetros del algoritmo genético

Parámetro	Valor
Tamaño de población (<code>pop_size</code>)	200
Generaciones máximas (<code>max_gen</code>)	500
Tasa de mutación (<code>mutation_rate</code>)	0.02
Tasa de cruzamiento (<code>crossover_rate</code>)	0.85
Elitismo (<code>elite_size</code>)	5
Tamaño de torneo (<code>tournament_size</code>)	5

Cuadro 5: Parámetros comunes a todas las variantes.

6. Análisis de Resultados Paralelos

6.1. Tabla completa de métricas (PC Windows – Ryzen 5600X)

La siguiente tabla consolida los resultados de las 405 ejecuciones (27 configuraciones $\times$ 15 semillas) sobre el equipo Windows. Se reportan tiempos promedio y desviación estándar, calidad de solución promedio y mejor valor factible, porcentaje de factibilidad, speedup y eficiencia paralela.

Cuadro 6: Métricas completas por variante, hilos e instancia (Ryzen 5600X, 15 reps.).

Variante	Hilos	Instancia	Avg (ms)	Std (ms)	Avg Valor	Std Valor	% Fact.	Speedup	Efic.
sequential	1	large	25 471,3	106,3	247 547	1486	100,0	1,000	1,000
sequential	1	medium	1866,5	8,9	25 620	289	100,0	1,000	1,000
sequential	1	small	241,5	1,4	3156	17	100,0	1,000	1,000
parallel	1	large	25 465,5	89,3	247 547	1486	100,0	1,000	1,000
parallel	1	medium	1863,4	8,7	25 620	289	100,0	1,002	1,002
parallel	1	small	242,6	1,6	3156	17	100,0	0,996	0,996
parallel	2	large	14 469,4	77,5	247 738	1125	100,0	1,760	0,880
parallel	2	medium	1094,8	6,5	25 671	321	100,0	1,705	0,852
parallel	2	small	168,0	1,3	3161	14	100,0	1,437	0,719
parallel	4	large	9168,9	60,4	246 774	987	100,0	2,778	0,695
parallel	4	medium	716,6	5,8	25 520	283	100,0	2,605	0,651
parallel	4	small	131,0	2,7	3159	14	100,0	1,844	0,461
parallel	8	large	7217,3	42,0	247 373	1154	100,0	3,529	0,441
parallel	8	medium	594,1	2,8	25 540	265	100,0	3,142	0,393
parallel	8	small	138,7	0,5	3162	14	100,0	1,742	0,218
islands	1	large	25 549,5	85,2	247 019	1092	100,0	0,997	0,997
islands	1	medium	1858,7	9,8	25 509	278	100,0	1,004	1,004
islands	1	small	243,9	2,1	3156	11	100,0	0,991	0,991
islands	2	large	26 245,4	124,6	251 471	1235	100,0	0,971	0,485
islands	2	medium	1917,2	11,5	25 943	205	100,0	0,974	0,487
islands	2	small	251,6	3,7	3169	11	100,0	0,960	0,480
islands	4	large	28 024,9	122,3	252 554	920	100,0	0,909	0,227
islands	4	medium	2055,1	18,7	26 218	165	100,0	0,908	0,227
islands	4	small	272,8	7,1	3173	9	100,0	0,885	0,221
islands	8	large	36 542,7	102,0	253 339	964	100,0	0,697	0,087
islands	8	medium	2725,9	56,4	26 399	156	100,0	0,685	0,086
islands	8	small	431,4	4,3	3181	6	100,0	0,560	0,070

Nota: Speedup = $T_{\text{seq_1hilo}}/T_{\text{variante}}$; Eficiencia = Speedup / hilos; % Fact. = porcentaje de ejecuciones con solución factible.

6.2. Tiempo de ejecución y speedup (variante parallel)

La tabla siguiente resume los tiempos promedio, desviación estándar y speedup para la variante **parallel** en la instancia **large** (10.000 ítems), que es donde el paralelismo tiene mayor impacto.

Hilos	Avg tiempo (ms)	Std tiempo (ms)	Speedup (S_p)	Eficiencia (E_p)
1	25.465	89,3	1,000	1,000
2	14.469	77,5	1,760	0,880
4	9.169	60,4	2,778	0,695
8	7.217	42,0	3,529	0,441

Cuadro 7: Speedup y eficiencia de la variante **parallel** en instancia **large** (Ryzen 5600X, 15 repeticiones por configuración).

El speedup crece de forma sublineal con el número de hilos. Al pasar de 1 a 2 hilos la eficiencia es alta (0,88), pero decae a 0,44 con 8 hilos. Esto se explica por la Ley de Amdahl: las secciones no paralelizadas —actualización del mejor individuo, elitismo, gestión de generaciones— representan una fracción constante del tiempo y limitan el speedup máximo teórico. Adicionalmente, el Ryzen 5600X dispone de 6 núcleos físicos, por lo que al usar 8 hilos se produce *oversubscription* con los hilos del sistema operativo, lo que reduce la ganancia incremental entre 4 y 8 hilos.

Para las instancias **medium** y **small** el speedup es menor (hasta 3,14 y 1,74 respectivamente con 8 hilos). En instancias pequeñas el overhead de OpenMP (creación y sincronización de hilos) es significativo respecto al cómputo útil, lo que reduce la eficiencia paralela.

Hilos	small		medium		large	
	S_p	E_p	S_p	E_p	S_p	E_p
1	0,996	0,996	1,002	1,002	1,000	1,000
2	1,437	0,719	1,705	0,852	1,760	0,880
4	1,844	0,461	2,605	0,651	2,778	0,695
8	1,742	0,218	3,142	0,393	3,529	0,441

Cuadro 8: Speedup y eficiencia de **parallel** por instancia (Ryzen 5600X).

6.3. Mejor fitness y calidad de solución

El **mejor fitness** reportado por el algoritmo corresponde al máximo valor de la función de aptitud $f(X)$ (incluyendo penalizaciones) alcanzado en toda la historia de evolución. En todas las ejecuciones experimentadas, el individuo con mayor fitness global coincidió con la mejor solución factible, ya que la calibración de penalizaciones ($\alpha = \beta = \delta = \varepsilon = 10 \times v_{\text{máx}} \times n$) garantiza que cualquier individuo inválido tenga fitness estrictamente negativo frente a cualquier solución válida con valor positivo. Por ello, el mejor fitness es equivalente al mejor valor factible reportado en las tablas.

La variante **parallel** no mejora la calidad de solución respecto a **sequential**, ya que ambas utilizan exactamente la misma lógica evolutiva sobre la misma población: la paralelización solo acelera la evaluación y generación de descendencia, pero no diversifica el espacio de búsqueda. La tabla siguiente muestra el valor promedio y la desviación estándar de la mejor solución factible para la instancia **large**:

Variante (Hilos)	Avg valor	Std valor	Mejor valor
sequential (1)	247.547	1.486	250.999
parallel (1)	247.547	1.486	250.999
parallel (2)	247.738	1.125	249.456
parallel (4)	246.774	987	248.404
parallel (8)	247.373	1.154	250.945
islands (2)	251.471	1.235	253.437
islands (4)	252.554	920	254.445
islands (8)	253.339	964	255.818

Cuadro 9: Calidad de solución para instancia **large** (Ryzen 5600X, 15 reps.).

El modelo de islas supera consistentemente al secuencial y al paralelo en calidad. Con 8 islas (8 hilos), el valor promedio aumenta un **+2,34 %** respecto al secuencial (253.339 vs 247.547) y el mejor valor encontrado en cualquier ejecución asciende a 255.818, superando en un **+1,9 %** al mejor del secuencial (250.999). Este beneficio se debe a la diversidad mantenida por la evolución independiente de subpoblaciones: cada isla explora regiones distintas del espacio de búsqueda y comparte periódicamente los mejores individuos mediante migración.

Cabe destacar que la variante **islands/1** (4 islas en 1 hilo) presenta un valor promedio ligeramente inferior al secuencial (247.019 vs 247.547), lo que se explica porque al correr las 4 islas en un solo hilo se ejecutan de forma secuencial con menor presupuesto de generaciones efectivas por isla.

6.4. Overhead del modelo de islas

El speedup del modelo de islas es inferior a 1 en todas las configuraciones, lo que indica que este modelo es **más lento** que el secuencial. Esto es esperado: el cómputo total es mayor (4 islas $\times$ 500 generaciones $\times$ 200 individuos en lugar de $1 \times 500 \times 200$) y el paralelismo reduce pero no elimina ese costo adicional. El objetivo del modelo de islas no es el tiempo sino la calidad.

Hilos	Speedup (islas vs seq)	Ganancia de calidad (large)
1	0,997	−0,21 %
2	0,971	+1,58 %
4	0,909	+2,02 %
8	0,697	+2,34 %

Cuadro 10: Relación entre speedup y ganancia de calidad del modelo de islas (large).

6.5. Porcentaje de soluciones factibles

En todas las configuraciones evaluadas, el 100 % de las 15 ejecuciones por grupo produjeron una solución factible (respecto a las restricciones duras: peso, volumen, incompatibilidades y dependencias). Esto valida que la calibración automática de penalizaciones ($\alpha = \beta = \delta = \varepsilon = 10 \times v_{\text{máx}} \times n$) es efectiva para guiar al algoritmo hacia regiones factibles del espacio de búsqueda.

7. Conclusiones

La implementación y análisis del algoritmo genético paralelo para el problema de la mochila extendida permitió extraer las siguientes conclusiones:

1. **Paralelismo efectivo en evaluación de aptitud.** La evaluación de la función de aptitud es la operación dominante en tiempo y es completamente paralelizable sin condiciones de carrera. Con 4 hilos se obtiene un speedup de 2,78 y con 8 hilos de 3,53 en la instancia grande, con eficiencias de 0,69 y 0,44 respectivamente.
2. **Limitaciones de la Ley de Amdahl.** La fracción secuencial del algoritmo (elitismo, actualización del mejor individuo, gestión de generaciones) limita el speedup máximo alcanzable. La reducción de eficiencia al aumentar hilos es consistente con este comportamiento, especialmente en instancias pequeñas donde el overhead de OpenMP es significativo.
3. **El modelo de islas mejora la calidad, no el tiempo.** La diversidad generada por subpoblaciones independientes con migración periódica produce soluciones un 2,34 % mejores en promedio que el algoritmo secuencial para la instancia grande, al costo de mayor tiempo de cómputo. La configuración de migración cada 25 generaciones con los 2 mejores individuos resultó efectiva para compartir buenas soluciones sin homogeneizar prematuramente las subpoblaciones.
4. **Robustez de la penalización.** La calibración automática de coeficientes de penalización basada en el valor máximo de la instancia garantizó el 100 % de soluciones factibles en todas las configuraciones experimentadas, validando la estrategia de distinguir restricciones duras y blandas.
5. **Reproducibilidad.** El uso de semillas fijas por configuración y el registro completo de parámetros en el CSV de resultados permite reproducir cualquier experimento de forma exacta.

Anexo

Resultados de otros miembros del grupo

Resultados Sebastian Cisternas

Resultados obtenidos en el equipo con las siguientes características:

- Sistema Operativo: Ubuntu 24.04
- Procesador: Ryzen 3 3200H
- Núcleos/Hilos: 2 Núcleos 4 Hilos
- RAM: 10 GB 2600 MHz

Cuadro 11: Métricas experimentales: rendimiento por variante (Islas, Paralelo, Secuencial) , hilos e instancia (100, 1000, 10000 items)

Variante	Hilos	Instancia	Avg (ms)	Std (ms)	Avg Valor	Std Valor	% Fact.	Speed-up	Efic.
islands	1	large	56 560,51	270,50	247 019,19	1092,00	100,0	0,994	0,994
islands	1	medium	4194,85	64,99	25 508,76	278,44	100,0	0,986	0,986
islands	1	small	602,94	24,99	3155,51	11,16	100,0	1,169	1,169
islands	2	large	81 518,22	593,16	251 470,85	1234,57	100,0	0,690	0,345
islands	2	medium	5935,90	82,90	25 943,18	205,30	100,0	0,697	0,349
islands	2	small	864,83	54,59	3168,99	11,05	100,0	0,815	0,408
islands	4	large	115 143,51	448,12	252 554,32	920,13	100,0	0,488	0,122
islands	4	medium	8609,48	151,66	26 218,25	164,51	100,0	0,481	0,120
islands	4	small	1463,22	51,37	3172,66	9,18	100,0	0,482	0,120
islands	8	large	206 187,28	1393,69	253 339,04	963,81	100,0	0,273	0,034
islands	8	medium	15 562,04	168,73	26 399,31	156,44	100,0	0,266	0,033
islands	8	small	2377,91	72,02	3181,16	6,06	100,0	0,296	0,037
parallel	1	large	56 249,88	283,14	247 547,33	1485,70	100,0	0,999	0,999
parallel	1	medium	4139,36	69,74	25 620,09	288,67	100,0	1,000	1,000
parallel	1	small	668,71	102,24	3156,16	17,07	100,0	1,054	1,054
parallel	2	large	41 730,33	1187,91	247 738,45	1125,26	100,0	1,347	0,674
parallel	2	medium	3233,66	189,41	25 671,00	320,67	100,0	1,280	0,640
parallel	2	small	564,18	71,85	3161,06	14,46	100,0	1,250	0,625
parallel	4	large	33 833,64	905,71	246 773,66	987,15	100,0	1,661	0,415
parallel	4	medium	2900,40	163,28	25 520,17	282,90	100,0	1,427	0,357
parallel	4	small	595,19	70,68	3159,01	13,79	100,0	1,184	0,296
parallel	8	large	31 949,03	238,86	247 372,84	1153,81	100,0	1,759	0,220
parallel	8	medium	2684,11	61,96	25 539,83	264,74	100,0	1,542	0,193
parallel	8	small	729,12	63,60	3162,07	14,38	100,0	0,967	0,121
sequential	1	large	56 214,00	286,51	247 547,33	1485,70	100,0	1,000	1,000
sequential	1	medium	4137,63	57,83	25 620,09	288,67	100,0	1,000	1,000
sequential	1	small	704,95	146,47	3156,16	17,07	100,0	1,000	1,000

Nota: Speed-up = $T_{\text{seq}}/T_{\text{variante}}$; Eficiencia = Speed-up / Hilos; % Fact. = porcentaje de ejecuciones con solución factible.

Cuadro 12: Speedup y eficiencia para la variante `parallel`

Instancia	Hilos	Speedup	Eficiencia
large	1	0,999	0,999
medium	1	1,000	1,000
small	1	1,054	1,054
large	2	1,347	0,673
medium	2	1,280	0,640
small	2	1,250	0,625
large	4	1,661	0,415
medium	4	1,427	0,357
small	4	1,184	0,296
large	8	1,760	0,220
medium	8	1,542	0,193
small	8	0,967	0,121

Cuadro 13: Calidad de solución ($\overline{\text{Valor}} \pm \text{StdValor}$) por variante, hilos e instancia

Variante	Hilos	Instancia	AvgValor	StdValor	MaxValor
islands	1	large	247 019	1092	248 788
islands	1	medium	25 509	278	25 956
islands	1	small	3156	11	3173
islands	2	large	251 471	1235	253 437
islands	2	medium	25 943	205	26 335
islands	2	small	3169	11	3188
islands	4	large	252 554	920	254 445
islands	4	medium	26 218	165	26 564
islands	4	small	3173	9	3188
islands	8	large	253 339	964	255 818
islands	8	medium	26 399	156	26 706
islands	8	small	3181	6	3188
parallel	1	large	247 547	1486	250 999
parallel	1	medium	25 620	289	26 241
parallel	1	small	3156	17	3183
parallel	2	large	247 738	1125	249 456
parallel	2	medium	25 671	321	26 186
parallel	2	small	3161	14	3178
parallel	4	large	246 774	987	248 404
parallel	4	medium	25 520	283	26 089
parallel	4	small	3159	14	3172
parallel	8	large	247 373	1154	250 945
parallel	8	medium	25 540	265	25 952
parallel	8	small	3162	14	3188
sequential	1	large	247 547	1486	250 999
sequential	1	medium	25 620	289	26 241
sequential	1	small	3156	17	3183

Cuadro 14: Ganancia de calidad de `islands` frente a `sequential` en la instancia `large`

Configuración	AvgValor (islands)	AvgValor (seq)	Ganancia (%)
Islands / 1 hilo	247 019	247 547	-0,21
Islands / 2 hilos	251 470	247 547	1,58
Islands / 4 hilos	252 554	247 547	2,02
Islands / 8 hilos	253 339	247 547	2,34

Resultados Christian Ferrer (Notebook)

Resultados obtenidos en el equipo con las siguientes características:

- Sistema Operativo: CachyOS (Arch Linux)
- Procesador: Intel Core i7 3537-u
- Núcleos/Hilos: 2 Núcleos 4 Hilos
- Ram: 8GB 1333 Mhz

Cuadro 15: Métricas experimentales: rendimiento por variante (Islas, Paralelo, Secuencial), hilos e instancia (100, 1000, 10000 ítems)

Variante	Hilos	Instancia	Avg (ms)	Std (ms)	Avg Valor	Std Valor	% Fact.	Speed-up	Efic.
islands	1	large	40 641,9	15 095,2	247 019	1092,0	100,0	0,994	0,994
islands	1	medium	2926,9	1068,4	25 509	278,4	100,0	0,999	0,999
islands	1	small	389,6	146,4	3156	11,2	100,0	0,991	0,991
islands	2	large	43 277,9	17 033,5	251 471	1234,6	100,0	0,934	0,467
islands	2	medium	3125,4	1208,3	25 943	205,3	100,0	0,935	0,468
islands	2	small	411,1	159,6	3169	11,0	100,0	0,939	0,469
islands	4	large	55 365,4	27 355,0	252 554	920,1	100,0	0,730	0,182
islands	4	medium	3974,8	1919,9	26 218	164,5	100,0	0,735	0,184
islands	4	small	580,3	307,7	3173	9,2	100,0	0,665	0,166
islands	8	large	100 270,1	63 729,2	253 339	963,8	100,0	0,403	0,050
islands	8	medium	7320,1	4595,6	26 399	156,4	100,0	0,399	0,050
islands	8	small	1104,8	673,5	3181	6,1	100,0	0,349	0,044
parallel	1	large	40 391,5	14 928,1	247 547	1485,7	100,0	1,001	1,001
parallel	1	medium	2924,3	1061,2	25 620	288,7	100,0	1,000	1,000
parallel	1	small	415,8	198,1	3156	17,1	100,0	0,928	0,928
parallel	2	large	23 965,0	9497,1	247 738	1125,3	100,0	1,686	0,843
parallel	2	medium	1787,2	703,7	25 671	320,7	100,0	1,636	0,818
parallel	2	small	256,9	93,9	3161	14,5	100,0	1,502	0,751
parallel	4	large	17 382,1	8214,0	246 774	987,2	100,0	2,325	0,581
parallel	4	medium	1329,2	612,8	25 520	282,9	100,0	2,199	0,550
parallel	4	small	217,4	88,5	3159	13,8	100,0	1,776	0,444
parallel	8	large	16 421,5	9204,6	247 373	1153,8	100,0	2,461	0,308
parallel	8	medium	1240,1	646,1	25 540	264,7	100,0	2,357	0,295
parallel	8	small	247,0	108,4	3162	14,4	100,0	1,563	0,195
sequential	1	large	40 415,0	14 946,6	247 547	1485,7	100,0	1,000	1,000
sequential	1	medium	2923,1	1056,9	25 620	288,7	100,0	1,000	1,000
sequential	1	small	386,0	144,5	3156	17,1	100,0	1,000	1,000

Nota: Speed-up = $T_{\text{seq}}/T_{\text{variante}}$; Eficiencia = Speed-up / Hilos; % Fact. = porcentaje de ejecuciones con solución factible.

Estructura del proyecto y ejecución

Estructura de carpetas

```
software3-a-genetico/
|-- src/
|   |-- main.cpp                # CLI, generador, punto de entrada
|   |-- instance_loader.hpp/.cpp # Carga de archivos CSV
|   |-- fitness.hpp/.cpp        # Funcion de aptitud y penalizaciones
|   |-- genetic_algorithm.hpp/.cpp # GA secuencial y paralelo (OpenMP)
|   |-- island_model.hpp/.cpp   # Modelo de islas
|-- data/
|   |-- small/      # 100 items
|   |-- medium/    # 1.000 items
|   |-- large/     # 10.000 items
|-- results/
|   |-- resultados.csv # salida raw por ejecucion (ignorado por git)
|   |-- metricas.csv  # metricas agregadas (ignorado por git)
|-- run_experiments.ps1 # disenno experimental completo (Windows/
    PowerShell)
|-- run_experiments.sh  # disenno experimental completo (Linux/WSL)
|-- show_results.ps1    # visualizador de metricas (Windows/PowerShell)
|-- show_results.sh     # visualizador de metricas (Linux/WSL)
|-- mochila_ga.exe      # ejecutable compilado (Windows)
|-- mochila_ga          # ejecutable compilado (Linux)
'-- .gitignore
```

Compilación

```
# Windows (MinGW-w64 / MSYS2)
g++ -O2 -fopenmp -std=c++17 src/*.cpp -o mochila_ga.exe

# Linux / macOS / WSL
g++ -O2 -fopenmp -std=c++17 src/*.cpp -o mochila_ga
```

Generar instancias

```
./mochila_ga --generate small --seed 42
./mochila_ga --generate medium --seed 42
./mochila_ga --generate large --seed 42
```

Ejecutar variantes individualmente

```
# Variante 1: secuencial (línea base)
./mochila_ga --instance data/small --variant sequential --seed 1

# Variante 2: paralelo con OpenMP
./mochila_ga --instance data/medium --variant parallel --threads 4 --seed
1

# Variante 3: modelo de islas
./mochila_ga --instance data/large --variant islands \
--threads 4 --islands 4 --mig-interval 25 --migrants 2 --seed
1
```

Diseño experimental completo (405 ejecuciones)

```
# Windows/PowerShell
.\run_experiments.ps1
.\run_experiments.ps1 -Instance small -Reps 3      # prueba rapida
.\run_experiments.ps1 -DryRun                      # ver sin ejecutar

# Linux/macOS/WSL
./run_experiments.sh
./run_experiments.sh --instancia small --reps 3
```

Calcular métricas y visualizar resultados

```
# Generar metricas.csv desde resultados.csv
./mochila_ga --analyze --results results/resultados.csv

# Visualizar (Windows)
.\show_results.ps1
.\show_results.ps1 -Instancia large

# Visualizar (Linux)
./show_results.sh --instancia large
```